Solution: Palindrome Check using Queue

Problem Statement

Given a string, determine if that string is a palindrome using a queue data structure.

A palindrome is a word, number, phrase, or other sequence of characters that reads the same forward and backward, ignoring spaces, punctuation, and capitalization.

Examples:

1. **Input**: "madam"
   Output: True
   Explanation: The word "madam" reads the same forwards and backwards.
2. **Input**: "openai"
   Output: False
   Explanation: The word "openai" does not read the same forwards and backwards.
3. **Input**: "A man a plan a canal Panama"
   Output: True
   Explanation: The phrase "A man a plan a canal Panama" reads the same forwards and backwards when we ignore spaces and capitalization.

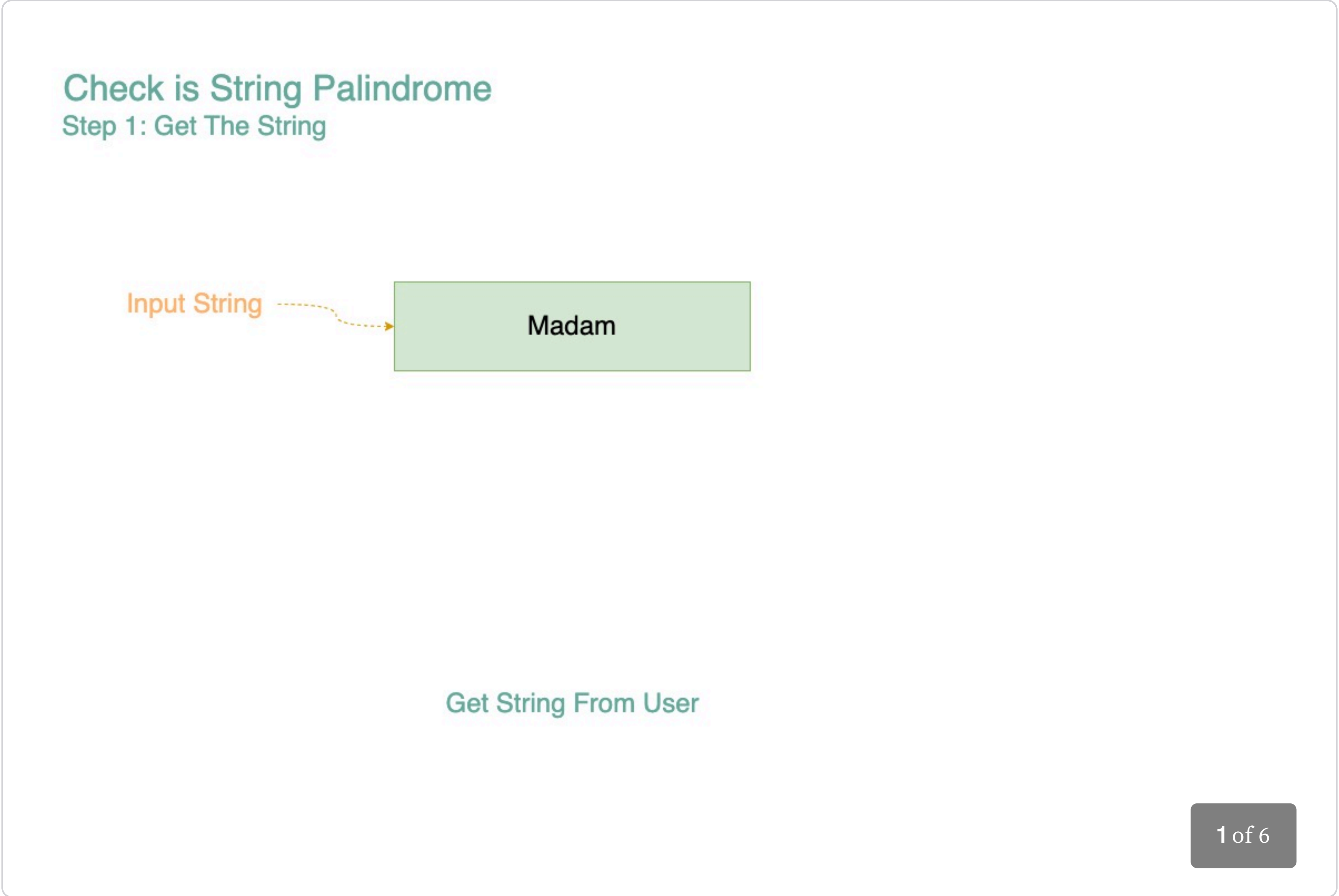
Solution

We will use a queue to solve this problem. The process will be to add each character from the string into the queue, and then dequeue each character from the front and the end, comparing them. If at any point the characters don't match, we can stop and return False. If we make it all the way through the string without finding a mismatch, then the string is a palindrome.

Algorithm

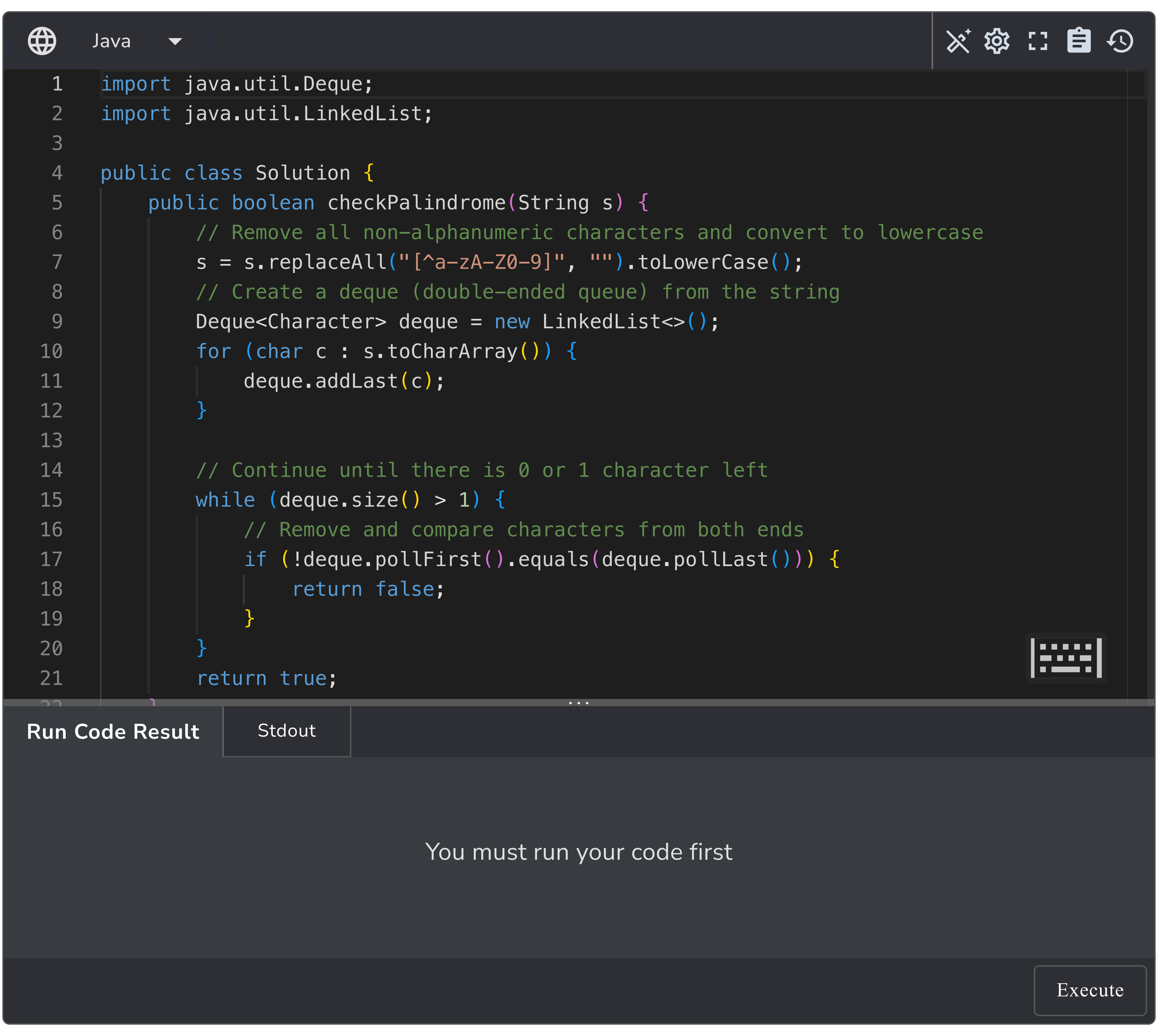
1. **Normalize the String:** Remove any spaces, punctuation, and convert all characters to the same case (lowercase or uppercase) to ensure consistency in comparison.
2. **Initialize a Queue:** Create an empty queue that will be used to store characters of the string. In Java, you can use `LinkedList` as a queue.
3. **Enqueue Characters:** Iterate over the normalized string and enqueue each character into the queue.
4. **Check for Palindrome:**
   - Dequeue a character from the front and end of the queue.
   - Compare these two dequeued characters.
   - If at any point the two characters do not match, return `false`, indicating the string is not a palindrome.
   - Repeat step-4, until there are less than 2 characters left in the queue.
5. **End of Queue:** If you've iterated through the queue without finding a mismatch, return `true`, indicating the string is a palindrome.

This algorithm efficiently uses a queue to compare characters from the beginning and end of the string, moving towards the center, to check for palindrome properties.



Code

Here is how we can implement this algorithm:



```java
import java.util.Deque;
import java.util.LinkedList;

public class Solution {
    public boolean checkPalindrome(String s) {
        // Remove all non-alphanumeric characters and convert to lowercase
        s = s.replaceAll("[^a-zA-Z0-9]", "").toLowerCase();
        // Create a deque (double-ended queue) from the string
        Deque<Character> deque = new LinkedList<>();
        for (char c : s.toCharArray()) {
            deque.addLast(c);
        }

        // Continue until there is 0 or 1 character left
        while (deque.size() > 1) {
            // Remove and compare characters from both ends
            if (!deque.pollFirst().equals(deque.pollLast())) {
                return false;
            }
        }
        return true;
```

You must run your code first

Time and Space Complexity

The time complexity of this algorithm is $O(n)$ where n is the length of the string because we need to iterate over the string to enqueue all characters. The space complexity is also $O(n)$ because we store all characters in a queue.

← Previous
Problem 4: Palindrome Check Using Queue

Next →
Problem 5: Zigzag Iterator

☐ Mark as Completed